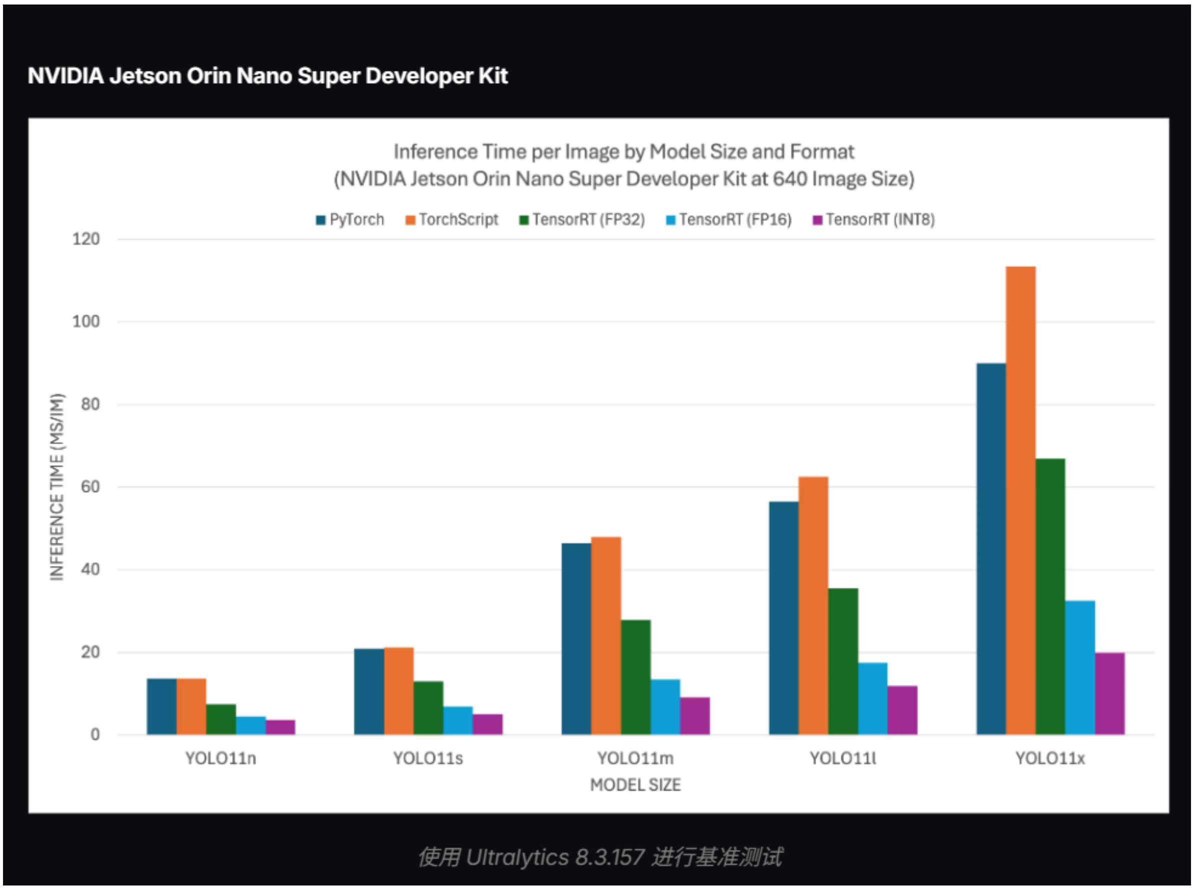
Model Conversion

1. YOLO (Benchmark)

Benchmark testing refers to the Ultralytics team's benchmark testing of YOLO on ten different model formats, measuring speed and accuracy. These formats include: PyTorch, TorchScript, ONNX, OpenVINO, TF SavedModel, TF GraphDef, TF Lite, MNN, NCNN, ExecuTorch. Since the official only provided comparison charts for yolo11 on JetsonOrin Super, we use yolo11's test results as reference here. It can be seen that TensorRT format models have the fastest inference speed.

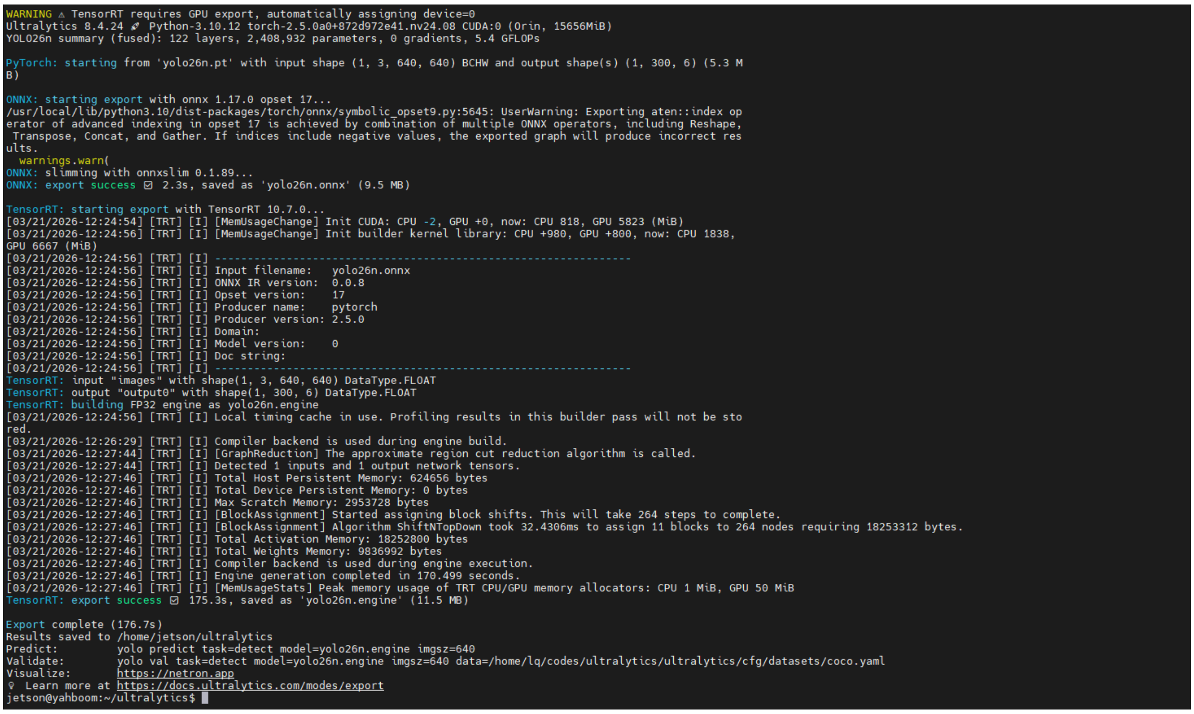
Comparison Chart



2. Model Conversion

Use yolo's CLI command in the terminal for conversion

```
yolo export model=yolo26n.pt format=engine # creates 'yolo26n.engine'
```



```
WARNING ⚠ TensorRT requires GPU export, automatically assigning device=0
Ultralytics 8.4.24 🚀 Python-3.10.12 torch-2.5.0a0+872d972e41.nv24.08 CUDA:0 (Orin, 15656MiB)
YOLO26n summary (fused): 122 layers, 2,408,932 parameters, 0 gradients, 5.4 GFLOPs

PyTorch: starting from 'yolo26n.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) (1, 300, 6) (5.3 M
B)

ONNX: starting export with onnx 1.17.0 opset 17...
/usr/local/lib/python3.10/dist-packages/torch/onnx/symbolic_opset9.py:5645: UserWarning: Exporting aten::index op
erator of advanced indexing in opset 17 is achieved by combination of multiple ONNX operators, including Reshape,
 Transpose, Concat, and Gather. If indices include negative values, the exported graph will produce incorrect res
ults.
  warnings.warn(
ONNX: slimming with onnxslim 0.1.89...
ONNX: export success ☑ 2.3s, saved as 'yolo26n.onnx' (9.5 MB)

TensorRT: starting export with TensorRT 10.7.0...
[03/21/2026-12:24:54] [TRT] [I] [MemUsageChange] Init CUDA: CPU -2, GPU +0, now: CPU 818, GPU 5823 (MiB)
[03/21/2026-12:24:56] [TRT] [I] [MemUsageChange] Init builder kernel library: CPU +980, GPU +800, now: CPU 1838,
GPU 6667 (MiB)
[03/21/2026-12:24:56] [TRT] [I] ----------------------------------------------------------------
[03/21/2026-12:24:56] [TRT] [I] Input filename:   yolo26n.onnx
[03/21/2026-12:24:56] [TRT] [I] ONNX IR version:  0.0.8
[03/21/2026-12:24:56] [TRT] [I] Opset version:    17
[03/21/2026-12:24:56] [TRT] [I] Producer name:    pytorch
[03/21/2026-12:24:56] [TRT] [I] Producer version: 2.5.0
[03/21/2026-12:24:56] [TRT] [I] Domain:
[03/21/2026-12:24:56] [TRT] [I] Model version:    0
[03/21/2026-12:24:56] [TRT] [I] Doc string:
[03/21/2026-12:24:56] [TRT] [I] ----------------------------------------------------------------
TensorRT: input "images" with shape(1, 3, 640, 640) DataType.FLOAT
TensorRT: output "output0" with shape(1, 300, 6) DataType.FLOAT
TensorRT: building FP32 engine as yolo26n.engine
[03/21/2026-12:24:56] [TRT] [I] Local timing cache in use. Profiling results in this builder pass will not be sto
red.
[03/21/2026-12:26:29] [TRT] [I] Compiler backend is used during engine build.
[03/21/2026-12:27:44] [TRT] [I] [GraphReduction] The approximate region cut reduction algorithm is called.
[03/21/2026-12:27:44] [TRT] [I] Detected 1 inputs and 1 output network tensors.
[03/21/2026-12:27:46] [TRT] [I] Total Host Persistent Memory: 624656 bytes
[03/21/2026-12:27:46] [TRT] [I] Total Device Persistent Memory: 0 bytes
[03/21/2026-12:27:46] [TRT] [I] Max Scratch Memory: 2953728 bytes
[03/21/2026-12:27:46] [TRT] [I] [BlockAssignment] Started assigning block shifts. This will take 264 steps to complete.
[03/21/2026-12:27:46] [TRT] [I] [BlockAssignment] Algorithm ShiftNTopDown took 32.4306ms to assign 11 blocks to 264 nodes requiring 18253312 bytes.
[03/21/2026-12:27:46] [TRT] [I] Total Activation Memory: 18252800 bytes
[03/21/2026-12:27:46] [TRT] [I] Total Weights Memory: 9836992 bytes
[03/21/2026-12:27:46] [TRT] [I] Compiler backend is used during engine execution.
[03/21/2026-12:27:46] [TRT] [I] Engine generation completed in 170.499 seconds.
[03/21/2026-12:27:46] [TRT] [I] [MemUsageStats] Peak memory usage of TRT CPU/GPU memory allocators: CPU 1 MiB, GPU 50 MiB
TensorRT: export success ☑ 175.3s, saved as 'yolo26n.engine' (11.5 MB)

Export complete (176.7s)
Results saved to /home/jetson/ultralytics
Predict:         yolo predict task=detect model=yolo26n.engine imgsz=640
Validate:        yolo val task=detect model=yolo26n.engine imgsz=640 data=/home/lq/codes/ultralytics/ultralytics/cfg/datasets/coco.yaml
Visualize:       https://netron.app
💡 Learn more at https://docs.ultralytics.com/modes/export
jetson@yahboom:~/ultralytics$
```

This command will convert models into two different formats: .onnx and .engine, where .onnx is a universal format

Besides yolo26n.pt, the four models for segmentation, pose detection, classification, and OBB can also be converted to .engine format using the same method.

```
yolo export model=yolo26n-seg.pt format=engine
yolo export model=yolo26n-pose.pt format=engine
yolo export model=yolo26n-obb.pt format=engine
yolo export model=yolo26n-cls.pt format=engine
```